# Projeto do Robô

Entrega 1 — Algoritmos

Grupo 06

**Integrantes**

Pedro

NOME DO COLEGA

Recursão pura, contagem de chamadas e regra gulosa

# Tarefa 1 — Descrição do problema

O projeto consiste em encontrar o caminho de menor custo para um robô que se movimenta em uma grade. O robô começa na posição (0, 0), no canto superior esquerdo, e precisa chegar ao canto inferior direito. Cada posição possui um custo inteiro positivo ou é um obstáculo, representado por `null` no JSON.

### Entrada

Um texto JSON com a chave `"custos"`, que guarda uma matriz de valores. Exemplo:

```
{"custos": [[2, 1], [3, 4]]}
```

### Saída

O programa devolve um dicionário com um destes estados:

• **ok**: existe caminho e o menor custo é devolvido;
• **sem_caminho**: a entrada é válida, mas não existe rota;
• **entrada_invalida**: o formato da entrada está incorreto, e vem uma mensagem de erro.

### Objetivo

Encontrar o menor custo possível para o robô chegar ao destino, usando recursão pura, sem memoização.

### Regras

1. O robô só anda para a direita ou para baixo, uma casa por vez.
2. Não pode sair dos limites da grade.
3. Não pode entrar em obstáculos.
4. O custo soma todas as casas visitadas, incluindo a partida e a chegada.
5. Os valores devem ser inteiros positivos ou `null`.
6. Todas as linhas devem ter o mesmo tamanho, e nenhuma pode ser vazia.

### Exemplo próprio 1 — entrada válida sem caminho

```
{"custos": [[2, 3, 1], [null, 4, null], [1, null, 5]]}
```

A entrada é válida, mas não existe rota até o destino. Do (0,0) o robô chega ao 4 do centro, mas dali a direita e a de baixo são obstáculos. Pelo outro lado, de (0,2) a casa de baixo também é obstáculo. Nenhum caminho chega ao canto inferior direito.

```
{"status": "sem_caminho", "custo": null}
```

### Exemplo próprio 2 — entrada inválida

```
{"custos": [[1, 2], [3]]}
```

A segunda linha tem menos elementos que a primeira, então a matriz não é retangular.

```
{"status": "entrada_invalida",
 "erro": "a linha 1 tem tamanho diferente"}
```

### Validação

A função `validar` usa `json.loads` dentro de `try/except` para detectar JSON quebrado. Depois confere o formato da matriz e os valores permitidos. Em Python, `True` é um tipo de `int` (`isinstance(True, int)` dá `True`). Por isso usamos `type(v) is int`, que rejeita `true`, além de `"1"`, `1.5`, `0` e `-1`.

## Tarefas 2 e 3 — Recursão e testes

A função `recursivo(mapa)` usa uma função interna `custo(i, j)`, que devolve o menor custo para sair da casa (i, j) e chegar ao destino:

• fora da grade ou obstáculo → `inf`;
• chegada → o custo da própria casa;
• caso contrário → custo da casa + `min(custo(i, j+1), custo(i+1, j))`.

No final, `inf` é convertido para `None`, que o `resolver` transforma em `sem_caminho`. Nenhum resultado é guardado entre as chamadas: não há dicionário, lista de resultados, variável global nem `@lru_cache`.

### Testes

Os testes usam `unittest` e rodam sozinhos com `python -m unittest discover -v`. A suíte tem **47 testes e todos passam**. Eles cobrem todos os casos da tabela do enunciado (custos 7, 6, 4, 11 e 6; os quatro casos sem caminho; as entradas inválidas) e mais casos criados pelo grupo, como grades 3×3 conferidas à mão, grade de uma casa, de uma linha e de uma coluna, e parede de obstáculos.

| Arquivo | Foco | Testes |
|---|---|---|
| test_validacao.py | entradas inválidas (as 11 do enunciado + casos do grupo) | 17 |
| test_resolver.py | ok, sem_caminho e casos do grupo | 18 |
| test_contagem.py | contagem de chamadas (tabela do enunciado + exemplo) | 6 |
| test_guloso.py | exemplo = 26, grade própria, preso, empate | 6 |

## Tarefa 4 — Por que funciona

### Por que a recursão termina?

De uma casa (i, j) faltam `(linhas-1-i) + (colunas-1-j)` movimentos até a chegada. Cada chamada recursiva anda uma casa para a direita ou para baixo, então falta **um movimento a menos**. Como esse número não pode diminuir para sempre, toda sequência de chamadas termina num caso base: a chegada, um obstáculo ou a saída da grade. Esses casos base não chamam ninguém, e por isso não existem chamadas infinitas.

### Por que encontra o menor custo?

Toda rota que sai de (i, j) começa indo para a direita ou para baixo. A melhor rota pela direita custa `custo(i, j+1)` e a melhor por baixo custa `custo(i+1, j)`, e isso vale para as casas menores pelo mesmo argumento (indução). Pegando o `min()` dos dois e somando o custo da casa atual, todas as rotas possíveis são consideradas e fica a mais barata.

### O que acontece quando não há rota?

Fora da grade e obstáculo devolvem `inf`. Como `min(inf, inf) = inf` e `custo + inf = inf`, a impossibilidade se propaga até a casa inicial. Se todos os ramos são impossíveis, o resultado final é `inf`, convertido para `None` e depois para `sem_caminho`. Se a partida ou a chegada for obstáculo, o resultado já é `inf` direto.

### Recursão sem memoização

O algoritmo não usa cache, dicionários ou tabelas. Uma mesma casa pode ser calculada várias vezes, e é isso que a próxima seção mede. A memoização fica para a Entrega 2.

## Tarefa 5 — Medição do trabalho

A função `contar_chamadas(mapa)` repete a mesma recursão, mas soma 1 num contador a cada chamada. O contador é uma variável externa, o que é permitido porque ele **conta** e não guarda resultados.

### Regra usada

Só chamamos o vizinho se ele estiver **dentro da grade** (testamos `j+1 < colunas` e `i+1 < linhas` antes de chamar). Assim, as chamadas que cairiam fora da grade não existem. Obstáculos que estão dentro da grade **são contados**, pois a chamada acontece e devolve `inf`.

### Resultados

| Grade | 2×3 | 3×3 | 6×6 | 10×10 | 1×10 | Exemplo |
|---|---|---|---|---|---|---|
| Chamadas | 9 | 19 | 923 | 184.755 | 10 | 43 |
| Esperado no enunciado | 9 | 19 | 923 | 184.755 | 10 | — |

Todos os números coincidem com o enunciado. As grades de 2×3 até 1×10 têm só casas de custo 1. A grade do exemplo não tem valor de referência no enunciado e deu **43** chamadas.

### Se a regra fosse outra

Se a função chamasse o vizinho antes de testar se ele está dentro da grade, as chamadas "fora da grade" também seriam contadas, e os números seriam maiores. Por exemplo, a 2×3 daria 13 em vez de 9, a 3×3 daria 27 em vez de 19 e a 1×10 daria 19 em vez de 10. A diferença são só chamadas que caem fora da grade e voltam `inf` na hora.

### Rastreio da grade 2×3 (9 chamadas)

Cada chamada aparece com a ordem em que acontece. Repare que a casa (1,1) é calculada **duas vezes** e a chegada (1,2) **três vezes**.

```
1. custo(0,0)
2.   custo(0,1)
3.     custo(0,2)
4.       custo(1,2)   chegada
5.     custo(1,1)
6.       custo(1,2)   chegada
7.   custo(1,0)
8.     custo(1,1)     (repetida)
9.       custo(1,2)   chegada (repetida)
```

### Por que cresce tão rápido?

A casa (1,1) pode ser alcançada vindo de (0,1) e vindo de (1,0). Cada vez que isso acontece, ela é recalculada do zero, junto com tudo que vem depois dela. Quanto maior a grade, mais caminhos passam pela mesma casa, e o número de chamadas cresce de forma **exponencial**. Nas grades só de 1s, o total é exatamente `C(L+C, L) − 1` (combinação). Por exemplo, na 10×10 dá C(20, 10) − 1 = 184.755. A memoização, na Entrega 2, resolve isso guardando cada casa uma vez só.

## Tarefa 6 — Regra gulosa

A função `guloso(mapa)` parte de (0,0) e, a cada passo, olha a casa da direita e a de baixo. Vai para a livre mais barata, prefere a direita no empate e nunca volta atrás. Se as duas estão bloqueadas antes da chegada, devolve `None`. É rápida (uma passada só), mas não garante o menor custo.

### Grade do exemplo: guloso 26 contra 11

```
(0,0)=2  total 2
direita 1 × baixo 3  → direita   total 3
direita 9 × baixo X  → direita   total 12
direita 9 × baixo 1  → baixo     total 13
direita 1 × baixo 1  → direita   total 14   (empate)
direita 9 × baixo X  → direita   total 23
baixo 2              → baixo     total 25
baixo 1 (chegada)    → baixo     total 26
```

O guloso dá **26**, e o ótimo (recursão) é **11**. O erro está no primeiro passo: o 1 da direita parecia melhor que o 3 de baixo, mas levou a uma linha cheia de 9 e a um obstáculo. O caminho ótimo aceita pagar 3 agora para descer até o corredor de 1s.

### Grade própria (3×3) em que o guloso erra

```
[[2, 1, 8],
 [3, 9, 9],
 [2, 1, 1]]
```

| Algoritmo | Caminho | Custo |
|---|---|---|
| Guloso | 2 → 1 → 8 → 9 → 1 | 21 |
| Ótimo (recursivo) | 2 → 3 → 2 → 1 → 1 | 9 |

**Onde a escolha atrapalhou:** na primeira casa, (0,0). Entre o 1 da direita e o 3 de baixo, o guloso escolheu o 1, que é mais barato naquele momento. Só que o 1 leva para a linha de cima, que termina em 8, e depois só resta descer por 9 e 1. O caminho de baixo custa 3 no primeiro passo, mas depois só passa por casas baratas (2, 1, 1). A regra "o mais barato agora" não enxerga o que vem depois.

O guloso também pode ficar preso. Em `[[1, null], [null, 1]]`, as duas casas vizinhas da partida são obstáculos, e a função devolve `None`.

## Conclusão

A recursão pura sempre encontra o menor custo, mas repete muito trabalho: 184.755 chamadas numa grade 10×10. O guloso faz uma passada só, mas pode errar bastante (26 contra 11 no exemplo e 21 contra 9 na grade própria). Os dois extremos motivam a Entrega 2: guardar os resultados de cada casa para ter a resposta certa sem o recálculo.